# Engagement Pulse | Engineering notes

Architecture, decisions and trade-offs • November 2026

A multi-tenant risk dashboard: consultants manage risks across assigned clients; viewers see only their organisation. I focused on access control, background imports and cross-instance updates.

## System overview

| |
|---|
| **Next.js / React**<br>Dashboard · React Query |
| REST / JWT + WebSocket |
| **Django / DRF / Channels**<br>Daphne ASGI · tenant checks · risk scoring |
| **PostgreSQL** — source of truth \| **Redis** — tasks + WS fan-out \| **Dramatiq** — CSV imports |

## Data model

**Organisation** → Projects → Risks; **Project** → CSV import jobs. A viewer belongs to one organisation; consultants use a user–organisation access mapping. Project risk score = min(100, Σ severity weight × status weight), recalculated transactionally.

## Three decisions

### 01 / Redis-backed live updates

**Why:** A client on instance B needs changes made through instance A.
**Choice:** Channels + Redis; publish data_changed after commit, then refetch.
**Alternatives:** Polling (delayed), SSE (viable), RabbitMQ (extra broker).
**Trade-off:** Notifications are best-effort.

### 02 / Async CSV imports

**Why:** CSV imports should not block HTTP requests.
**Choice:** Dramatiq + Redis; persisted job status and frontend polling.
**Alternatives:** Synchronous (blocks API), Celery (richer workflows), RQ (basic Redis queue).
**Trade-off:** Shared upload volume works locally, not for distributed production.

### 03 / Shared DB with tenant-scoped access

**Why:** Consultants need multi-client access without exposing client data.
**Choice:** PostgreSQL ownership plus backend checks for APIs, WS and imports.
**Alternatives:** Per-tenant DBs (operational cost); row-level security (future layer).
**Trade-off:** Every new endpoint needs tenant-access tests.

## Security choices

Access JWT in memory; refresh token in an HttpOnly cookie. WebSockets use 30-second, single-use Redis tickets plus an independent organisation access check. No long-lived JWT in the WS URL.

# Verification, limits and next steps

## What I checked

**75 backend tests · 30 frontend tests** passed. Docker smoke checks covered login, tenant isolation, WebSocket handshake and CSV completion. I also verified cross-instance delivery: a risk update through port 8000 produced `data_changed` on a viewer socket connected to a second Daphne instance on port 8001.

## What I deliberately left out

Browser E2E automation and CI; some project/risk management UI flows; production monitoring and audit history. The backend supports broader CRUD than the current UI. This is a working local implementation, not a production deployment.

## What I would do next, in order

**1. Reliability** — Add Playwright smoke tests and GitHub Actions; instrument logs, job failures and WebSocket connections.

2. Operational readiness · Add SHA-256 file fingerprints to deduplicate CSV imports; move uploads to object storage; add queue metrics, retry policies, backups and audit trails.

**3. Scale when needed** — Run more Daphne and Dramatiq replicas independently; use managed PostgreSQL/Redis, connection pooling and query profiling. Add an outbox if event delivery guarantees become important.

**4. Product depth** — Complete project/risk creation flows and add filtering, pagination and score history.

## How I used AI

I used Google Antigravity for bounded implementation, tests and debugging. `AGENTS.md` set the stack, security rules and Git controls. I chose the architecture and priorities, reviewed changes, ran tests and manually verified the two-instance WebSocket path. I stopped when the scope exceeded the intended timebox rather than presenting E2E/CI as done.

## Design boundaries

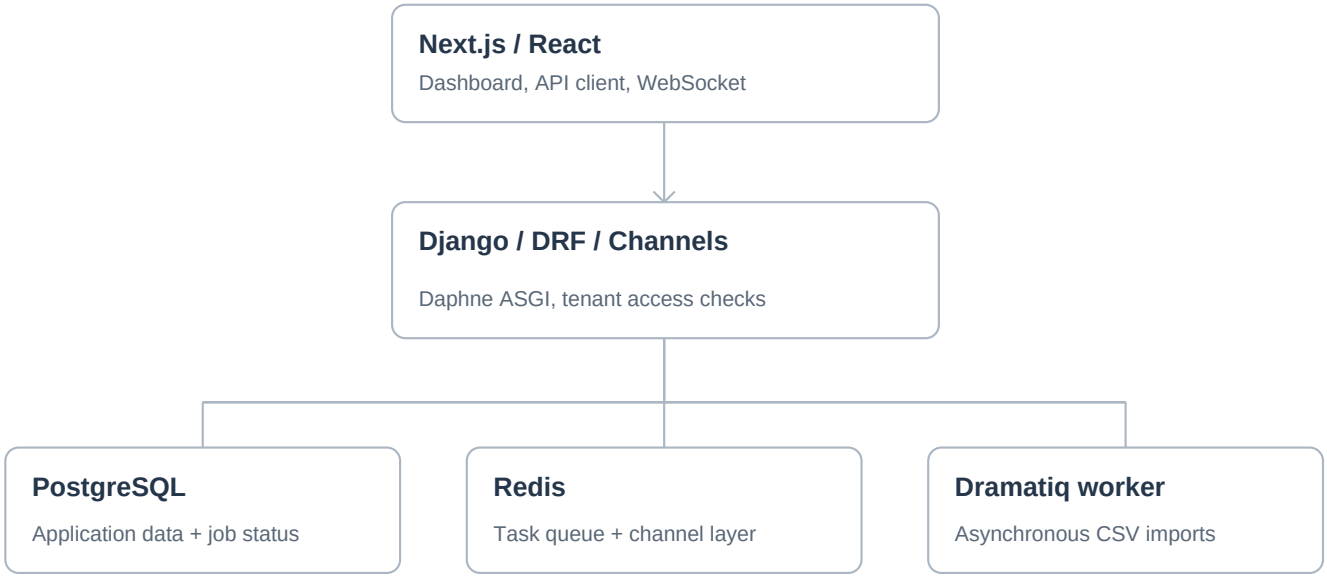
PostgreSQL owns state; WebSocket messages are invalidation hints, not copies of business data. The backend enforces tenant access even if the UI hides actions. Redis is shared infrastructure for queues and fan-out, not the system of record. I would not split into microservices without a clear scaling or ownership reason.

See README.md for setup and demo accounts; AGENTS.md for engineering guidance. Azure deployment and Entra ID migration are covered in a separate note.

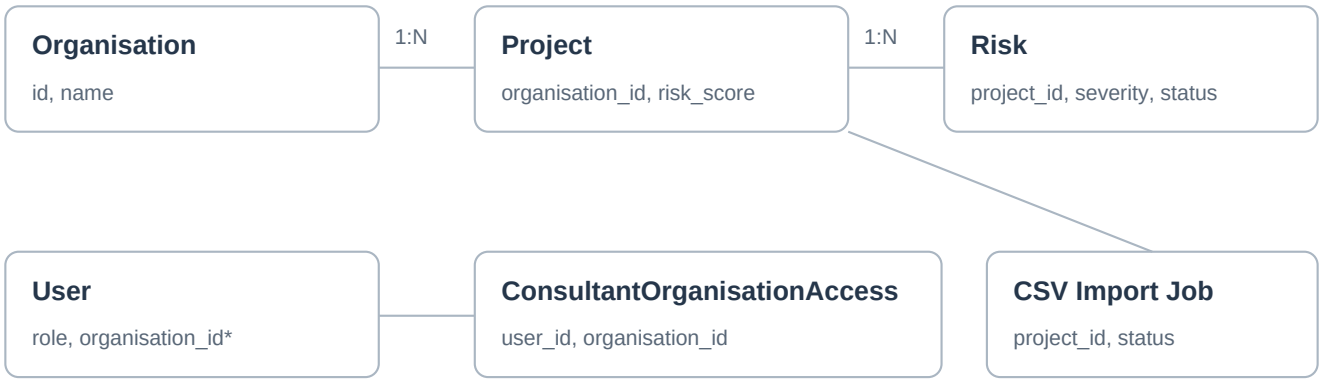
# Architecture and data model

Simple reference diagrams for the local Docker Compose setup

## Service architecture




## Database relationships




* Viewers belong to one organisation; consultants use the access mapping.

Conceptual relationships; fields are illustrative, not a full schema dump.